■ Palmas Ride Engine

How it works · How it decides · How it learns

Methodology Document

A practical guide to the data sources, scoring formula, rain detection, self-calibration, and accuracy-tracking that decide whether to ride Alto de Palmas this morning — or tomorrow morning.

Updated June 11, 2026

1. Overview

Palmas Ride Engine is a single-purpose web app that answers one question: **should I bike up Alto de Palmas at 5:00 AM?** It blends multiple meteorological sources, weights them through a transparent scoring formula tailored to the Las Palmas climb, and produces a 0–100 score, a YES/NO decision, a confidence level, and a list of human-readable reasons.

Beyond the live prediction, the app keeps an evolving accuracy record: every prediction is logged the night before, and the next morning the observed weather is cross-validated from two independent sources and written back so that, over time, the model's calibration is visible to the rider.

A self-calibrating shadow model runs silently alongside the main formula. It trains on the growing history of night-before predictions and ground-truth actuals, converging over several weeks from a simple threshold learner to Platt-scaled logistic regression. Its verdict is shown on the main card as a 'second opinion' without replacing the transparent scoring formula.

Why this exists

Open-Meteo and the SIATA weather network each have blind spots when applied to a microclimate-prone road that climbs from the Aburrá Valley (~1,500 m) to Alto de Palmas (~2,500 m). Generic forecasts often under-predict rain at altitude; SIATA stations down in the valley don't always reflect what's actually falling on the upper sections of the climb. The engine combines them deliberately so neither source alone can dictate the call.

What it is not

This is not a general weather forecast. It is narrowly tuned to the 05:00–07:30 morning ride window at Alto de Palmas, Medellín. The corridor filter, scoring weights, and time conventions are all built around that specific use case.

2. System Architecture

The app is split into four cooperating components running on Vercel.

Layer	Implementation	Role
Frontend	Static HTML / vanilla JS / Leaflet	Renders the main card, the map, the radar, and the history view. Installa
Backend	Python on Vercel Functions (@vercel/python)	Fetches data from SIATA + Open-Meteo, computes the score, exposes
Storage	Vercel Blob (private store)	predictions.json — append-mostly log of every prediction + its observed
Scheduler	Vercel Cron Jobs (twice daily)	Locks in tomorrow's prediction at 20:00 Bogota and backfills yesterday's

Request flow (live view)

```
Browser -> GET /api/check
|-- fetch SIATA Pluviometrica.json (corridor stations)
|-- enrich: per-station {code}.json (parallel x 15)
|-- fetch SIATA radar animation JSON
|-- fetch SIATA WRF forecast (Centro + Envigado)
|-- fetch Open-Meteo forecast (hourly precip + wind)
|-- fetch Open-Meteo air quality (PM2.5, PM10, AQI)
|-- compute score, decision, reasons
|-- apply shadow-model calibration (calibration.json)
'-- best-effort save to predictions.json (Vercel Blob)
|
v
Browser renders main card + shadow verdict + (optionally) More Details panel.
```

Request flow (cron)

Vercel Cron sends authenticated GET requests to /api/cron at fixed times. The 20:00 run predicts tomorrow's ride; the 07:00 run backfills the just-finished ride's actual and retrains the shadow model. Both writes are idempotent.

3. Data Sources

Five real-time inputs feed the engine. Each is queried fresh on every call (with a short in-memory cache so consecutive visits within five minutes share data).

Source	Endpoint	What we use
SIATA pluvio (summary)	Pluviometrica.json	Per-station current valor (rainfall snapshot).
SIATA pluvio (detail)	{code}.json	p10m, p1h, p24h rainfall totals + sensor freshness.
SIATA radar	animacion_radar.json	Last ~12 frames of precipitation echoes + bounds, animated in the Mo
SIATA WRF	wrfmedCentro.json + wrfenvigado.json	Local 24-hour rainfall forecast by quarter-day (LOW / MEDIA / ALTA /
Open-Meteo forecast	forecast?hourly=...	Hourly precip, precip probability, temperature, humidity, wind speed fo
Open-Meteo air quality	air-quality?current=...	PM2.5, PM10, US AQI, European AQI. Informational only.

SIATA is the *Sistema de Alerta Temprana de Medellín y el Valle de Aburrá* — the regional weather-and-hazards network. Their public JSON endpoints are unauthenticated but use self-signed SSL, so we relax the certificate check in the fetcher.

4. The Route Corridor — which stations count

SIATA publishes hundreds of stations across the Aburrá Valley. Most are irrelevant to whether the road up Las Palmas is wet. The first job of the engine is therefore to narrow the SIATA feed to stations whose readings actually represent Palmas weather.

How we define "on the climb"

We pulled the live geometry of **Avenida de Las Palmas** (OSM ref=56) and **Variante Las Palmas** from OpenStreetMap via the Overpass API. The result is 186 ordered segments containing 1,883 waypoints, covering the road from its El Poblado start to past Alto de Palmas.

A SIATA station qualifies for the corridor if the minimum great-circle distance from its (lat, lon) to any route waypoint is at most **1.5 km**. With waypoints this dense, that radius cleanly captures stations directly on or immediately adjacent to the road and excludes stations on the other side of the ridge.

In practice

At publication time, 15 stations pass the corridor filter. The top three by distance from the road are Colegio Latino (0.04 km, literally on Av. Las Palmas), ISAGEN (0.08 km), and Q. La Sanin (0.20 km). The farthest in-corridor station sits ~1.4 km off-road.

Why 1.5 km, not 2.5 km? An earlier iteration used a square bounding box around the climb. That included off-route stations like Pan de Azucar (4 km north) and Las Flores in Guarne (east, past the summit) whose rain is essentially uncorrelated with Palmas. The distance-from-route filter with dense OSM waypoints lets us tighten the radius substantially without losing any genuinely-on-Palmas station.

5. The Riding Window, Timezone & Time-aware Framing

The score is computed for a fixed window: **05:00–07:30 Bogota local time**. All scoring inputs are aggregated over that window only (with one exception: "overnight precipitation", which uses the hours leading up to 05:00).

Why Bogota time matters

Vercel functions run in UTC. Bogota is UTC−5 with no daylight saving. A naive datetime.now() on the server disagrees with the rider's wall clock by 5 hours, which matters at day boundaries. All date arithmetic goes through a small Bogota timezone shim (timeutil.py) that anchors today_str() and tomorrow_str() to UTC−5.

Time-aware framing: "this morning" vs "tomorrow morning"

The app is opened at two very different points in the ride cycle: the **night before** (~20:00–midnight, planning) and **pre-dawn** (~04:00, about to leave). Before the engine had time-aware framing, both opens always scored *tomorrow's* ride — so a 04:00 open showed a prediction ~25 hours away rather than the one ~1 hour away.

The fix is a soft cutoff at **08:00 Bogota** — one hour after the ride window closes. Before 08:00 the *target date* is today and the framing is **"this morning"**; at or after 08:00 the target is tomorrow and the framing is **"tomorrow morning"**. This is not just a label change: the engine genuinely re-scores today's imminent window with the latest data.

```
# timeutil.py additions
MORNING_CUTOFF_HOUR = 8

def target_date_str():
if now_bogota().hour < MORNING_CUTOFF_HOUR:
return today_str() # framing = this_morning
return tomorrow_str() # framing = tomorrow_morning

def framing():
return 'this_morning' if now_bogota().hour < 8 else 'tomorrow_morning'

def evening_before(date_str):
# Returns the calendar day before date_str — used to anchor the
# overnight-precip window to 20:00(date-1) → 04:59(date).
d = datetime.strptime(date_str, '%Y-%m-%d') - timedelta(days=1)
return d.strftime('%Y-%m-%d')
```

Canonical vs morning logging slots

The night-before prediction is the **canonical entry** — frozen and graded by the calibration system. A pre-dawn re-score writes only to a separate entry["morning"] slot (save_prediction(morning=True)) and never touches the canonical score, decision, or correct field. The shadow model ignores the morning slot entirely — it trains only on canonical entries.

This also fixes a cron interaction: the 07:00 Bogota run has framing "this_morning", so its predict step writes only the observation slot; the graded night-before prediction is never clobbered.

6. Scoring Formula

Every prediction starts at 100. Each input either applies a penalty (subtracts points) or a bonus (adds points). The final score is clamped to the range 0–100.

Penalties

Trigger	Threshold	Penalty
Average rain probability (05:00–07:30 from Open-Meteo)	> 60 %	–40
Any corridor SIATA station reports active rain	valor ≥ 0.1 mm OR p10m ≥ 0.1 mm OR p1h ≥ 0.5 mm	–50
Max wind in the window	> 20 km/h	–15
Average humidity in the window	> 90 %	–10
Roads classified 'wet' (overnight precip > 5 mm OR active rain)	—	–15
Roads classified 'damp' (overnight precip 1–5 mm)	—	–7 (half)

Bonuses

Trigger	Threshold	Bonus
Fully dry forecast window (every hour: precip < 0.1 mm AND prob < 50 %)	all hours	+25
No corridor station reporting rain (and at least one online)	—	+10

Decision & confidence

```
final_score = clamp(0, 100, sum of penalties + bonuses)
decision = YES if final_score >= 50 else NO
confidence = high if score >= 70
medium if 40 <= score < 70
low if score < 40
```

What is intentionally NOT in the score

- **WRF forecast** — fetched and shown in the reasons list, but contributes no points. It's information-only.
- **Radar imagery** — fetched, displayed in More Details, but never weighted. It's a visual aid.
- **Air quality (PM2.5, PM10, AQI)** — fetched and shown with a non-zero color badge, but explicitly disclaimed as "not factored into the score."

The score is a heuristic, not a calibrated probability. A score of 100 does not mean a 100% chance of dry roads. The empirical mapping from score to actual dry-ride probability is what the accuracy log is gradually filling in, and what the shadow model is learning to approximate.

7. Rain Detection — Four Signals

A naive 'is it raining now' check on SIATA's pluviometric data was responsible for two real-world misses: a sub-millimeter zombie reading from one station permanently anchored the score at 50, and a real rainy Saturday morning slipped past entirely because the summary valor field reported zero between brief showers.

The fix was to stop treating valor as the only signal. Every corridor station now exposes four independent rainfall signals; a station is flagged 'raining' if any of them crosses its threshold:

Signal	Window	Source	Raining threshold
valor	current snapshot	Pluviometrica.json	≥ 0.1 mm
p10m	last 10 min	{code}.json	≥ 0.1 mm
p1h	last 1 hour	{code}.json	≥ 0.5 mm
p24h	last 24 hours	{code}.json	informational only (used for road-wetness)

Sub-mm noise threshold

Values below 0.1 mm are treated as no rain. SIATA's pluvio stations regularly report tiny non-zero residuals (e.g. 0.02 mm) on dead sensors that haven't been re-zeroed. Below this threshold we ignore the reading rather than let one zombie sensor anchor the entire score.

Sensor-liveness cross-check

Even with the 0.1 mm threshold, SIATA could in principle serve a stale 0.5 mm residual from a sensor whose rainfall hardware is actually offline. The cross-check fetches the station's per-station detail file: if all three of p10m, p1h, and p24h are –999, the rain sensor is treated as offline and the value is dropped.

Detail files are fetched in parallel for all 15 corridor stations using a ThreadPoolExecutor with a 3-second per-station timeout. Total enrichment latency is typically ~0.7 s. Each file is cached for 5 minutes.

8. Road Condition Inference

Road wetness is computed separately from 'is it raining' because the surface state at 5:00 AM depends on what fell overnight, not on what's falling at the moment of the prediction (8:00 PM the night before). Two independent sources are combined:

Source	Window	Used as
Open-Meteo overnight precip	20:00 evening_before(target) → 04:59 target (Europe/Madrid)	Sum of hourly precip
SIATA p24h average	rolling last 24 h across corridor	Mean of corridor stations' p24h

The overnight window is anchored to evening_before(target_date) so that it correctly covers the previous night regardless of whether the prediction is for today (this morning) or tomorrow (tomorrow morning).

The worst case wins. The two sources are combined by taking max(open_meteo_overnight, siata_p24h_avg) and bucketing the result:

```
max(om_overnight, siata_p24h_avg) -> road condition
> 5 mm -> wet
1-5 mm -> damp
0.2-1 mm -> mostly_dry
<= 0.2mm -> dry
```

If SIATA captures rain that Open-Meteo's interpolated grid missed — the classic Palmas microclimate situation — SIATA wins and the road is flagged damp or wet accordingly. The reverse also works.

9. Recording the Actual — Dual-Source Backfill

Each prediction is logged at 20:00 Bogota the night before. The morning after the ride window closes, the engine records what actually happened, using the same two sources blended the same way:

```
open_meteo_precip = sum(forecast.past_days['precipitation']
for hours 05:00-07:00 of the ride day)
siata_p24h_avg = mean(corridor stations' p24h at 07:00 cron time)
# p24h covers ~yesterday afternoon -> 07:00 =
# the ride window plus overnight
actual.precip_mm = max(open_meteo_precip, siata_p24h_avg)
actual.rained = actual.precip_mm > 0.1
```

Source label & disagreement

Every backfilled actual carries an explicit source field:

- **agreed** — both sources within 0.5 mm of each other.
- **open_meteo** — Open-Meteo recorded more rain than SIATA.
- **siata_p24h** — SIATA recorded more rain than Open-Meteo's grid. These are the interesting ones: cases where the microclimate showed up at the on-road stations but missed the broader forecast model.

Why 'max' instead of average?

When the two sources disagree by a meaningful amount, one of them is almost always closer to ground truth. The expected error of averaging is symmetric, but for a cyclist, under-reporting is the worse failure mode. Taking the max biases toward over-reporting, which is the safer asymmetry to carry.

10. The Audit Trail

Every prediction in predictions.json stores a complete snapshot of the sensor state at the time of the prediction and at the time of the backfill. This lets a reader inspect any past entry and see exactly what each station was reporting, and what each external source said about the morning in question.

Schema (one entry)

```
{
"ride_date": "2026-06-11",
"predicted_at": "2026-06-10T01:00:00Z",
"score": 92, "decision": "YES", "confidence": "high",
"reasons": ["Moderate/low rain probability (8%)", ...],
"forecast": {
"avg_precip_prob": 8.0, "max_wind": 4.2,
"avg_humidity": 96.0, "overnight_precip_mm": 0.3,
"station_snapshots": [ ... one per corridor station ... ]
},
"morning": { // pre-dawn re-score only
"score": 89, "decision": "YES", "confidence": "high",
"at": "2026-06-11T09:15:00Z"
},
"actual": { // filled in the next morning by cron
"precip_mm": 5.1, "rained": true,
"open_meteo_precip_mm": 0.0, "siata_p24h_avg_mm": 5.1,
"source": "siata_p24h",
"station_snapshots": [ ... ]
},
"override": { // set via the History UI if needed
"rained": true, "note": "Poured until 7am", "set_at": "..."
},
"correct": false // YES + rained = wrong call
}
```

The morning slot holds the pre-dawn re-score as observational data. It never overwrites the canonical score/decision/correct fields, so it does not affect the calibration system or accuracy statistics.

Source labels surface directly in the History view as small badges (Open-Meteo / SIATA / Agreed) so a SIATA badge on a wrong prediction is the visual signal that microclimate rain was the specific failure mode.

11. Automation

Two Vercel Cron Jobs guarantee the daily prediction and backfill happen whether anyone visits the site or not. Both hit the same endpoint (/api/cron), which always runs both jobs — they're idempotent.

Bogota	UTC cron	What it does
20:00	0 1 * * *	Locks in tomorrow's prediction. Framing = 'tomorrow_morning'. Writes canonical entry; als
07:00	0 12 * * *	Backfills the just-finished ride's actual (dual-source). Framing = 'this_morning' so predict s

Cron security

/api/cron is gated by CRON_SECRET, a strong random value stored as a Vercel environment variable. Vercel attaches Authorization: Bearer ${CRON_SECRET} to its outbound cron requests; the endpoint verifies and returns 401 otherwise.

Why redundant logging?

/api/check and /api/history also opportunistically save predictions and backfill actuals — mirroring exactly what cron does. Writes are upserts keyed by ride_date and refuse to overwrite once an actual is recorded, so the redundancy is safe.

12. Self-calibrating Shadow Model

The scoring formula is transparent and human-tunable, but its decision boundary (threshold = 50, penalties = fixed values) was set by intuition, not by data. The shadow model runs silently alongside it: it trains on the growing history of canonical night-before predictions and ground-truth actuals and gradually learns a data-driven second opinion.

The shadow model's output appears on the main card as a secondary line ("El modelo aprendido diría: NO (62% lluvia)") without replacing the transparent score. Once enough data accumulates, it can be promoted to the primary decision-maker.

Staged learner

The model advances through three stages as labelled history accumulates. Each stage requires a minimum number of examples with at least one positive (rained) and one negative (dry) label:

Stage	Algorithm	Unlocks at	Description
0 — Threshold	LOO score threshold	≥ 5 labelled	Simple threshold on the canonical score. Leave-one-out CV finds
1 — Platt	Platt scaling (LOO)	≥ 10 labelled	Fits a logistic sigmoid to the raw score: P(rain) = σ(a·score + b).
2 — Logistic	L2 logistic regression (LOO)	≥ 20 labelled	Multi-feature logistic regression on (avg_precip_prob, max_wind,

Balanced class weights

Dry mornings far outnumber rainy ones. Without correction, a naive learner maximises accuracy by always predicting dry. The training uses **balanced class weights**: each rainy example is up-weighted by n_total / (2 × n_rained) and each dry example by n_total / (2 × n_dry). Balanced accuracy (mean([TPR, TNR])) is also the metric reported in the calibration panel so the display does not reward 'always say dry' gaming.

Leave-one-out cross-validation

With fewer than 30 examples, k-fold CV wastes too much of the training set. All three stages use LOO: for each candidate hyperparameter, train on all *n–1* examples and score the left-out one. The parameter with the best mean balanced accuracy across all folds is selected. This gives a conservative but honest generalization estimate on a small dataset.

apply_calibration output

```
calibrate.apply_calibration(cal_state, score, features) -> {
'shadow_decision': 'YES' | 'NO' | None,
'shadow_prob_rain': 0.0 .. 1.0 | None,
'shadow_basis': 'threshold' | 'platt' | 'logistic' | 'gathering',
'stage': 0 | 1 | 2 | None
}
```

When shadow_basis == 'gathering' the calibration panel shows a progress counter (e.g. '3 of 10 needed for Platt') so the rider knows how many more rides the learner needs before upgrading.

Persistence

The trained state is stored in calibration.json in the same Vercel Blob store as predictions.json. It is retrained automatically on every cron run and opportunistically whenever /api/history is called and

the stored state is more than 24 hours old.

13. UI Tour

Main view

The primary card shows the **framing label** ("Esta mañana" or "Tomorrow morning" depending on time of day), the vibe line, the numeric score with a gradient bar, confidence, the 05:00–07:30 riding window, the inferred road conditions, and the human-readable reasons that fed the score. If the shadow model has data, a secondary line shows its verdict and estimated rain probability.

Actions row

Below the main card sit three action buttons:

- **Refresh** — re-fetches live data, bypassing the in-memory cache.
- **Share** — triggers the native Web Share API (WhatsApp and others) with a framing-aware localized message. Falls back to a direct wa.me link on browsers that don't support the Share API.
- **Install** — appears on Android Chrome when the PWA install prompt is available; tapping it adds the app to the home screen with its own icon.

More Details

A toggle expands the detail panel, which is purely informational: air quality (PM2.5/PM10/AQI), animated SIATA radar overlaid on the OSM map, and the Stations on the Climb mini-map with colored pins and four-signal pop-ups.

History

Stats card (accuracy %, correct, wrong, pending) plus a list of recent predictions. Each row is expandable to show the full prediction details, source badge, and override buttons ("It rained" / "It was dry" / "Clear"). The calibration panel (accuracy grid — raw and balanced) appears at the bottom of the History view when the shadow model has run at least one complete LOO pass.

Language toggle

A circular EN/ES button fixed to the top-right corner toggles the UI between English and Colombian Spanish. The choice persists in localStorage. Dates use the matching locale (en-US vs es-CO). Backend-generated reasons currently stay in English regardless.

PWA install + offline

The app ships a manifest.json and service worker (sw.js). The service worker uses a **cache-first** strategy for the app shell (HTML, CSS, JS, icons, Leaflet) and **network-only** for all /api/* routes so predictions are always live when the network is available.

On offline or spotty-signal opens, the last successful /api/check response is read from localStorage and rendered with an explicit **"offline — last updated HH:MM"** banner. This directly serves the 04:00 pre-dawn spotty-signal case: the rider sees the cached call rather than a blank error screen.

On **iOS**: add to home screen via Safari's Share → 'Add to Home Screen'. On **Android Chrome**: an Install button appears in the actions row when the browser fires a beforeinstallprompt event.

14. Refresh & Caching Semantics

Three caching layers exist; understanding them prevents the surprise of 'I clicked refresh but nothing changed.'

Layer	TTL	Bypassed by
Service worker (app shell)	Until CACHE_VERSION bumped	Bump the version string; old cache evicted on activate.
In-memory cache (cache.py, per Vercel serverless instance)	5 minutes (300 s)	Refresh button (?fresh=1) clears it explicitly.
Vercel Blob CDN (in front of predictions.json)	Up to 60 s	Etag-based cache-busting query string: ?v=<etag>

The Refresh button is the user-facing escape hatch: clicking it sends a request with ?fresh=1 and cache: 'no-store'. The server clears its in-memory cache before fetching, and the blob read uses the etag cache-buster so the just-written prediction is visible immediately.

15. Limitations & Known Trade-offs

- **SIATA detail files are rolling.** Their per-station endpoints expose only p10m / p1h / p24h — there's no archive. So we cannot retroactively reconstruct a past day's rain from SIATA.
- **Backend-generated reasons are English-only.** The score formula returns fully-formed sentences. Localizing them would require refactoring analyze.py to return structured keys and formatting on the frontend.
- **The score is heuristic, not probabilistic.** A score of 100 is not P(dry) = 1.0. The shadow model provides calibrated probabilities, but it is a second opinion, not the primary output.
- **Shadow model needs ~20 labelled days to reach logistic regression.** Until then it runs on the threshold or Platt stage. The calibration panel shows a progress indicator.
- **Vercel Hobby tier function timeout is 10 s.** Current /api/check runtime is ~5 s in the worst case (15 parallel SIATA detail fetches + Open-Meteo + radar JSON). There's headroom but not infinite.
- **WRF and radar are not in the score.** They are shown to the rider but never weighted. A future refinement could parse WRF's morning rain levels into a small penalty.

These are deliberate boundaries, not unfixed bugs. Each one is a tradeoff: more sources = more latency; full localization = more code surface; calibrated probabilities require labelled data we are still accumulating.

16. Future Work

- **Promote shadow model to primary.** Once the logistic stage is live and balanced accuracy consistently exceeds the fixed-formula accuracy, offer a UI toggle to use it as the main decision.
- **Reliability diagram.** Plot predicted probabilities (from shadow model) vs. empirical rain frequency in each probability bucket. A well-calibrated model's dots sit on the diagonal.
- **Per-source accuracy breakdown.** Which signal predicts best — Open-Meteo's precip probability, SIATA's overnight readings, the WRF forecast? Splitting accuracy by source would reveal which inputs carry the weight.
- **WRF in the score.** Add a small penalty for MUY ALTA / ALTA in the morning forecast, since it currently contributes nothing to the numeric score.
- **Push notification at 20:30 Bogota.** When the cron finishes the 20:00 prediction, fire a Web Push notification so the rider doesn't have to open the site.
- **Multi-day forecast view.** Show a 3-day strip of yes/no/uncertain days so route planning can look beyond tomorrow.
- **Best-window scan.** If the canonical window (05:00–07:30) is borderline, find the driest 2-hour sub-window within a wider range.
- **Backend reason localization.** Return structured keys with params instead of pre-formatted sentences, so the EN/ES toggle covers the reasons too.
- **Other climbs.** The corridor filter, scoring, and calibration pipeline are generic. El Escobero or Altavista would need only a new route geometry and station set.

Appendix A — Configurable Knobs

All of these live in config.py and can be adjusted without touching the rest of the codebase.

Knob	Default	What it controls
RIDE_EARLIEST	5	First hour of the ride window (Bogota)
RIDE_LATEST	7	Hour the ride window ends (07:00–07:59 inclusive)
MORNING_CUTOFF_HOUR	8	Before this hour: target = today ('this morning')
CORRIDOR_RADIUS_KM	1.5	Max km a station can sit off-route to count
SCORING.rain_prob_high_threshold	60 %	Threshold above which the rain-probability penalty fires
SCORING.rain_prob_high_penalty	–40	Penalty applied above the threshold
SCORING.active_rainfall_penalty	–50	Applied when any corridor station reports rain
SCORING.high_wind_threshold	20 km/h	Above this max wind triggers the wind penalty
SCORING.high_wind_penalty	–15	Applied above the wind threshold
SCORING.high_humidity_threshold	90 %	Above this avg humidity triggers the humidity penalty
SCORING.high_humidity_penalty	–10	Applied above the humidity threshold
SCORING.dry_window_bonus	+25	All hours in window must be precip < 0.1 mm AND prob < 50 %
SCORING.no_recent_rain_bonus	+10	No corridor station reporting rain
SCORING.wet_road_penalty	–15	Applied when roads are inferred wet (half for damp)
CACHE_TTL_SECONDS	300	How long the in-memory cache holds each SIATA / OM response

Appendix B — HTTP Endpoints

Endpoint	Method	Auth	Purpose
/api/check	GET	none	Compute & return the prediction for target_date (today before 08:00, tomor
/api/history	GET	none	Return the prediction log + accuracy stats. Side effect: opportunistic backfil
/api/override	POST	Bearer palmas	Set or clear a manual ground-truth for a ride_date. Body: {ride_date, rainec
/api/cron	GET	Bearer CRON_SECRET	Cron-only endpoint. Always logs tomorrow's prediction AND backfills pendi
/api/health	GET	none	Simple liveness probe.

Repository

github.com/bensykora997/palmasridechecker

File map

```
config.py # Tuning knobs (corridor radius, scoring weights)
palmas_route_data.py # Avenida de Las Palmas geometry from OSM
timeutil.py # Bogota tz shim + target_date/framing/evening_before
fetch_siata.py # SIATA pluvio + radar + WRF
fetch_openmeteo.py # Open-Meteo forecast + archive
fetch_air.py # Open-Meteo air quality
analyze.py # Score + road conditions + station analysis
calibrate.py # Self-calibrating shadow model (pure Python)
cache.py # 5-minute in-memory cache
db.py # Vercel Blob client (predictions.json + calibration.json)
api/check.py # GET /api/check
api/history.py # GET /api/history
api/override.py # POST /api/override (Bearer palmas)
api/cron.py # GET /api/cron (Bearer CRON_SECRET)
api/health.py # GET /api/health
static/ # index.html + app.js + style.css (vanilla + Leaflet)
static/manifest.json # PWA manifest
static/sw.js # Service worker (cache-first shell, network-only /api/*)
static/icons/ # icon-192/512/maskable, apple-touch-icon, favicon.svg
docs/build_icons.py # Dev-only Pillow icon generator
vercel.json # Routes + cron schedule
```

■ Buenas pintas y a clavar, parcero.